

Neural Scene Representation for Locomotion on Structured Terrain

David Hoeller^{1,2}, Nikita Rudin^{1,2}, Christopher Choy², Animashree Anandkumar^{2,3}, Marco Hutter¹

Abstract—We propose a learning-based method to reconstruct the local terrain for locomotion with a mobile robot traversing urban environments. Using a stream of depth measurements from the onboard cameras and the robot’s trajectory, the algorithm estimates the topography in the robot’s vicinity. The raw measurements from these cameras are noisy and only provide partial and occluded observations that in many cases do not show the terrain the robot stands on. Therefore, we propose a 3D reconstruction model that faithfully reconstructs the scene, despite the noisy measurements and large amounts of missing data coming from the blind spots of the camera arrangement. The model consists of a 4D fully convolutional network on point clouds that learns the geometric priors to complete the scene from the context and an auto-regressive feedback to leverage spatio-temporal consistency and use evidence from the past. The network can be solely trained with synthetic data, and due to extensive augmentation, it is robust in the real world, as shown in the validation on a quadrupedal robot, ANYmal, traversing challenging settings. We run the pipeline on the robot’s onboard low-power computer using an efficient sparse tensor implementation and show that the proposed method outperforms classical map representations.

Index Terms—Representation Learning; Deep Learning for Visual Perception

I. INTRODUCTION

HUMANS and animals are endowed with a strong intuition about their physical environments. We rely on the spatial model we have formed from past experience to overcome common terrains with minimal mental effort. We can walk up a flight of stairs after a brief initial glance and can switch focus away from the stairs. Similarly, getting an accurate terrain map from few scans is a crucial skill that robots need for locomotion and navigation, yet computing such a 3D map is a challenging problem due to noisy sensors, occlusions, drifts in localization, etc. [1], [2], [3], [4]. Recent advances in deep learning have shown that it is also possible to train robust robotic systems from past data and synthesize the experience in a single neural network [5]. In locomotion

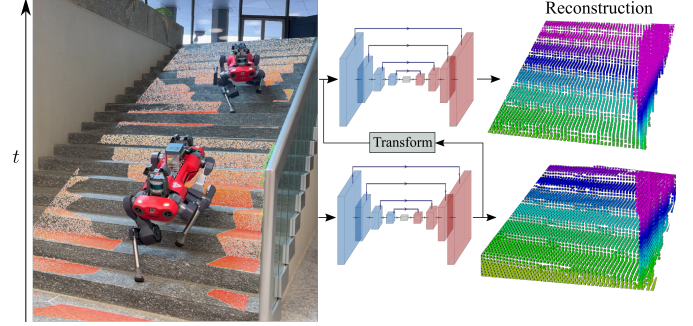


Fig. 1: Overview of our approach. The noisy point cloud obtained from the depth sensors and the previous output are passed through the network, producing a point cloud estimate of the surrounding scene. Due to the auto-regressive nature of the approach, the reconstruction is refined over time, and the network remembers objects that have entered the robot’s blind spots.

research, they have enabled unmanned ground vehicles such as legged robots to operate under harsh conditions [6].

However, walking on rough terrain is still challenging for quadrupedal robots. The robot has to estimate the terrain using exteroceptive sensors, and the locomotion controller needs to make sense of this high-dimensional information. In many cases, exteroceptive sensors such as cameras and depth sensors suffer from large amounts of motion blur, changing lighting conditions, and occlusions. To make matters worse, commercially available quadrupeds such as ANYmal or Spot have a specific depth camera set-up that leaves blind spots in the critical areas for proper foothold placement, such as below the robot. Thus, mobile robots must first build a map using point cloud measurements and odometry information. The data is fused over time to estimate the terrain around the robot. The surface is then passed on to the locomotion controller to choose the correct footholds to overcome the rough environment. Due to the harsh conditions, heavy drifts in localization, and noisy depth observations, the reconstructed maps tend to be noisy and unfit for locomotion. As a result, established methods such as Elevation Mapping [3] or Voxblox [7] need to rely on many hyper-parameters and heuristics, requiring expert knowledge for tuning. In [6], the authors address these issues and deploy a locomotion policy that internally updates the map estimate by combining the sparse height scans with proprioceptive sensing using a recurrent structure. However, the map representation is tied to the controller and cannot be used by another module.

To tackle these challenges, we present a learning-based

Manuscript received: February, 24, 2022; Revised: May, 15, 2022; Accepted: June, 06, 2022.

This paper was recommended for publication by Editor Jens Kober upon evaluation of the Associate Editor and Reviewers’ comments.

This work was supported by NVIDIA, the Swiss National Science Foundation (SNSF) through project 188596, the National Centre of Competence in Research Robotics (NCCR Robotics), and the European Union’s Horizon 2020 research and innovation program under grant agreement No.780883. Moreover, this work has been conducted as part of ANYmal Research, a community to advance legged robotics.

¹ Affiliated with the Robotic Systems Lab, ETH Zürich, Switzerland

² Affiliated with NVIDIA

³ Affiliated with Caltech, USA

Correspondence: dhoeller@ethz.ch

algorithm that estimates the local terrain around the robot from a stream of noisy depth measurements, see Fig. 1. The pipeline takes the robot's pose, the partial observations from the cameras, and the latest map estimate as input and reconstructs the scene, even in the occluded regions. The autoregressive feedback allows the network to maintain the detailed geometry using the information from the previous inputs. We train the network to build an intuition about the spatial configuration of the scene from the current context, similar to what humans do. When walking on stairs, for example, despite only observing the steps partially, the module understands the situation and extends the stairs below the robot, even when a large portion of the surface is occluded. Our network also uses the temporal sequence to memorize completely occluded objects that could be seen in past measurements, such as when walking over a box.

We use NVIDIA's IsaacGym [8] simulation environment to create a large number of randomized scenes with walls, roadblocks, flights of stairs, and boxes. Our robotic agents move around the scene to collect a data-set consisting of more than 200,000 point cloud observations. During training, we simulate the depth camera noise and apply extensive data augmentations making the 3D reconstruction network predict the ground-truth environment despite excessive noise and incomplete observations. Our module is trained on synthetic data only, and due to the augmentations, it transfers sim-to-real and generalizes to real-world sensory data, as we show in our experiments in various environments.

While we do not claim to solve the complete 3D rough terrain problem and focus on the simpler urban setting with structure, this is a promising step towards that direction. To summarize, this paper presents the following contributions:

- A novel approach that reconstructs urban-like environments under the harsh requirements of the real world, i.e. noisy point clouds, missing data from stereo matching failures, noisy state estimation from jerky motions and impacts with the world resulting in imprecise trajectory estimates.
- A method to initialize the map with a meaningful guess without heuristics using the context inferred from the visible data.
- An evaluation in simulation and on the real robot, showing that the approach can handle state estimator drifts and large amounts of missing data from readily-available depth cameras and outperforms currently used baselines that rely on heuristics.

II. RELATED WORK

a) *Locomotion*: Legged locomotion is a well-studied field of robotics. While past works mainly focus on model-based control [9], learning-based methods have increasingly come to the spotlight due to recent advances in deep reinforcement learning [5]. Blind quadrupedal locomotion on flat terrain using a neural network is demonstrated in [10] and [11]. The authors in [12] and [13] build upon these works and show that it is to some extent possible to walk on rough terrain without exteroceptive sensors with a quadruped and a biped,

respectively. Newer approaches show successful locomotion on more challenging terrain using perceptive inputs [6], [14], [15]. A carefully designed architecture is used in [6] to switch between a blind controller and a perceptive one depending on the quality of the terrain reconstruction and heavily noisify the terrain measurements during training. In [16], the policy uses a transformer to process proprioceptive and exteroceptive data to perform locomotion and obstacle avoidance. However, similar to [6], the vision model is tied to the policy and has to be retrained for every new policy. In the experiments, we show that our method works with cheap and readily available cameras and that it can be used as a standalone module by different model-based and learning-based policies without retraining.

b) *Environment Mapping*: Mapping the 3D environment using exteroceptive sensing has been studied in the context of robotics. The three most commonly used representations are 2.5D elevation maps [1], [2], [4], [3], point clouds [17], [18], [19], and voxel grids [20], [7].

The elevation map representation is widely used in robotics for locomotion [21], [6], [14] and planing [22] due to its simplicity. The terrain is encoded as a top-down view image around the robot. In [3], the authors use a probabilistic formulation and fuse the range measurements and the robot's pose using a Kalman filter to produce an estimate of the height profile of the surrounding terrain. The main drawback is that it is susceptible to drift in odometry. Also, height maps cannot represent the scene in full 3D, and a table, for example, is represented as one single block. The authors in [23] extend the approach and perform hole filling on the height map with a neural network. The method does not perform filtering in the visible regions and since it only considers the current time step, it also suffers from drifting issues.

Point cloud-based methods are mainly used in the context of Simultaneous Localization and Mapping (SLAM). Such systems use RGB or range information to produce an estimate of the robot's trajectory and incrementally build a map of the world. PTAM [17] is a lightweight parallel mapping and tracking algorithm that uses a sparse set of image correspondences. Semi direct [18] and direct [19] odometry frameworks use photometric errors to map the environment densely and estimate the relative poses instead of feature-based matching.

Lastly, voxelized-based mapping has also been used for 3D reconstruction. OctoMap [20] is an octree-based hierarchical probabilistic 3D voxel environment representation. This representation is efficient and fast, but only uses voxel-wise binary occupancy. Voxblox [7] is a similar volumetric mapping library that uses voxel hashing [24] to grow the environment dynamically and saves distance to the closest surface in the form of a Euclidean Distance Transforms (EDT).

Most of these 3D mapping algorithms rely on classical methods to reconstruct the scene and thus lack the capabilities to semantically complete missing information. We use a voxel-based approach, where each voxel stores the relative coordinates of the corresponding point to achieve sub-voxel accuracy. Unlike previous works, we use a learning-based algorithm to complete unseen parts of the scene using a neural network and handle drift in odometry.

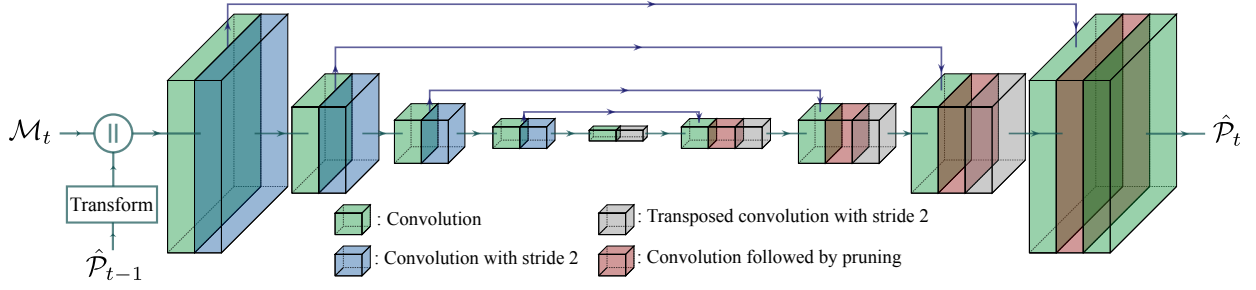


Fig. 2: Architecture of the terrain representation module. The previous point cloud estimate $\hat{\mathcal{P}}_{t-1}$ at time $t-1$ is first transformed into the current frame and concatenated temporally with the current point cloud measurement $\mathcal{M}_t$ at time t . The result is fed through a fully convolutional encoder-decoder network with skip connections to produce an estimate of the current point cloud $\hat{\mathcal{P}}_t$ at time t .

c) **3D Scene Completion:** In many cases, commercial 3D scanners fail to reconstruct the geometry of the scene accurately due to various factors such as registration failure and occlusion. Such incomplete 3D scans can cause errors in the subsequent parts of the pipeline and thus, many recent works have proposed learning-based 3D scene completion from partial/incomplete 3D scans. Song et al. [25] proposed voxelized dense 3D scene completion with semantic labels. Dai et al. [26] use similar dense 3D voxelized 3D completion but add an auto-regressive hierarchical structure to create a high-resolution completion. In VolumeFusion [27], the authors take a sequence of RGB images to estimate the depth maps and fuses them with the images' features to estimate the truncated signed distance function of the scene, but has no proper recurrence to keep information from the past. Sun et al. [28] propose a 3D reconstruction network that completes a scene using a hierarchical spatially sparse neural network. The network up-samples voxels and prunes unnecessary ones to reduce the computational cost of high-resolution reconstruction. This is similar to Gwak et al. [29] who employ generative convolutions and pruning. We adopt a similar approach of up-sampling and pruning voxels to generate high-resolution 3D reconstructions while reducing the computational complexity.

III. METHODOLOGY

This section describes how we reconstruct the terrain from noisy and occluded observations. The pipeline is depicted in Fig. 2. Using the pose difference between the previous time step and the current one, the previous point cloud estimate $\hat{\mathcal{P}}_{t-1}$ (output of the network) is first transformed to the current measurement frame. The result is concatenated with the current measurement $\mathcal{M}_t$, and fed into a fully convolutional network, producing the point cloud estimate $\hat{\mathcal{P}}_t$.

A. Input Pre-processing

First, the point cloud has to be converted to a data structure that can be forwarded through the network. We choose a voxel grid representation, where the voxels store the relative coordinates of the points to achieve sub-voxel accuracy. Since a dense grid representation has cubic complexity and could take up a large amount of memory, we use a sparse formulation.

To achieve this, we discretize the current point cloud measurements into a $64 \times 64 \times 64$ grid that represents a $3.2\text{m} \times 3.2\text{m} \times 3.2\text{m}$ map around the robot. For each voxel, we define a feature as the offset between the centroid of the points that fall within that voxel and the voxel's bottom left rear corner. Mathematically, let $\mathbf{p}_i = [x_i, y_i, z_i] \in \mathbb{R}^3$ be the centroid point at the i -th occupied voxel in the grid's reference frame, i.e. one unit is equal to one cell. The sparse input tensor in coordinate list format (COO) maps the discretized cell coordinate c_i of that centroid to a feature value f_i , which are defined as

$$\begin{aligned} c_i &= [\lfloor x_i \rfloor, \lfloor y_i \rfloor, \lfloor z_i \rfloor, k_i], \\ f_i &= \mathbf{p}_i \bmod 1 \end{aligned} \quad (1)$$

where mod is the modulo operator, and k is the discrete time index of that point. k is set to 0 for the points of the current measurement and to 1 for the points from the previous output. This introduces a temporal component to the problem and lets the network perform 4D convolutions across space and time. Due to the modulo operation, the feature values represent the normalized coordinates of the centroid in each voxel and are in the range $[0, 1]$. The continuous centroid location can be retrieved by adding the cell index with the corresponding feature value.

B. Architecture

The network is a U-Net-like [30] fully convolutional 4D encoder-decoder network with skip connections. This architecture allows the decoder to maintain the fine details from the encoder.

The encoder is a sequence of convolutions that down-sample the input by a factor of 16 for each spatial axis. It uses 4 strided convolutions, each down-sampling the coordinates by a factor of 2 spatially but preserving the temporal dimension. As a result, the network keeps the two temporal channels separate and performs convolutions across space and time at the different resolutions.

The decoder uses the latent tensor as input. Additionally, the feature maps of each encoder block are forwarded to the corresponding decoder block using skip connections. The sequence of convolutions up-sample the tensors back to the same stride as the input. However, since the latent tensor is

likely to be a fully occupied block of dimension $4 \times 4 \times 4 \times 2$, standard up-sampling would produce a fully occupied voxel grid with $64 \times 64 \times 64 \times 1$ cells. This would become prohibitively slow and would thwart the training process due to the sparsity of the problem. Therefore, similar to [29], we introduce pruning operations in the decoder. This step discards elements from the feature maps before passing the data to subsequent stages. Specifically, at each layer, we take the features from the current sparse tensor T_f to generate a likelihood for each voxel using a separate convolutional layer and a sigmoid activation, resulting in a sparse tensor T_p . We prune all the elements of T_f whose likelihood in T_p is smaller than a constant $\alpha \in [0, 1]$. The tensor T_p is not forwarded to the next layers.

The final convolution of the decoder maps the features to a dimension of 3 to produce an estimate of the sub-voxel position of the points, as in Eq. 1.

C. Training Procedure

The target for reconstruction is the ground truth point cloud transformed according to Eq. 1. The loss function is composed of two parts. The first one computes the binary cross-entropy between the features of T_p at each layer and a value of 1 or 0, depending on whether an occupied voxel is present at these coordinates in the target at the resolution of that layer. The other part takes the mean Euclidean distance between the output features and the features of the target to estimate the sub-voxel position of the points. Due to that formulation, at a given coordinate of T_p , the feature value represents the network's confidence that the voxel is occupied.

The pruning threshold α is a key hyper-parameter that can be modified to change the behavior of the network. When α is high, the network is conservative and reconstructs only the points it is confident about. However, this reduces the output density, and holes can appear in the reconstruction. When α is reduced, the output becomes denser, but the network generates points not necessarily in the target. We found experimentally that setting α to 0.5 results in a good trade-off between the estimate's recall and precision.

To train the network, we roll it out on a sequence of 12 time steps and compute the loss at each step. We found experimentally that propagating the gradients over time did not provide any benefit on reconstruction performance. We use the Adam optimizer with an initial learning rate of 0.01 that is exponentially decayed until 0.0001.

D. Data Generation

The architecture described in the previous subsection can estimate the ground truth point cloud around the robot from the stream of noisy point cloud measurements. To further encourage the module to reconstruct scenes with structure, we collect a large data set in urban-like settings. As a result, the network overfits to such environments and tries to implicitly identify key parameters of the scene, such as the length and height of stairs from the noisy data.

We generate structured environments consisting of stairs, boxes on the ground, walls, poles, and narrow corridors in

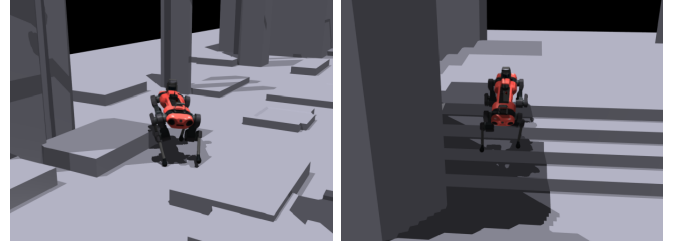


Fig. 3: Randomized environments generated in simulation. The parameters of the scene are sampled uniformly: the stairs' width and height between $[0.2, 0.5]$ m and $[0.08, 0.25]$ m, respectively; the boxes' width and length between $[0.2, 2.0]$ m and their height between $[0.08, 0.25]$ m. The walls are sampled to produce corridors of width in the range $[2, 6]$ m.

simulation (Fig. 3) using NVIDIA's IsaacGym [8]. The parameters of the scene's elements, such as the boxes' dimensions or the walls' locations, are randomized to generate a data set that reflects the real world's diversity.

The robot is controlled towards a reachable position with a randomized speed and base orientation using a rough terrain locomotion policy. The measurements are provided by four simulated Intel RealSense depth cameras placed at the front, back, left, right of the robot and tilted downwards by 30° , which is the standard configuration on the ANYmal C robot. The ground truth point clouds are obtained by sampling a dense point cloud from the mesh of the terrain around the robot at every time step. On average, 43% of the ground truth points are visible in the measurements in the data-set comprising of 200000 time steps.

E. Data augmentation

During training, the measurements are noisified (Fig. 4) to make the pipeline robust to the noise of the real system and facilitate sim-to-real transfer. We use the following augmentations:

- **Position:** The position of each point is disturbed uniformly in the range $[-0.05 \text{ m}, 0.05 \text{ m}]$
- **Tilt:** The point cloud is tilted in a random direction by an angle sampled uniformly in the range $[-1^\circ, 1^\circ]$
- **Height:** The height of random patches of the point cloud is disturbed uniformly in the range $[-0.05 \text{ m}, 0.05 \text{ m}]$
- **Pruning:** Random patches of the point cloud are removed
- **Outliers:** Random clusters of points are added to the measurement
- **Robot Pose:** The position of the robot is uniformly disturbed in the range $[-0.05 \text{ m}, 0.05 \text{ m}]$

We also randomly mirror the data along the x and y axes for a whole trajectory to produce more diverse trajectories.

The network is therefore trained for denoising and completion and has to utilize the evidence from previous measurements to estimate the partially observable state of the world.

The backbone of our pipeline relies on the Minkowski Engine [31], which provides CPU and GPU accelerations for neural networks for spatially sparse tensors.

IV. EXPERIMENTS

We validate our approach with various experiments in simulation and on the real robot and compare against the baselines Elevation Mapping [3] and Voxblox [7]. While both baselines have more than ten hyper-parameters and heuristics, our approach only has the pruning threshold α that needs tuning. We report the mean precision, recall, F1 score by discretizing the world in a robot-centric voxel grid of dimension $64 \times 64 \times 64$ with cell sizes $0.05 \text{ m} \times 0.05 \text{ m} \times 0.05 \text{ m}$, as well as the mean the absolute height difference between the reconstruction and the ground truth. The latter is essential for the locomotion task since the policy directly uses the height information.

A. Evaluation in Simulation

The output of our module on a validation trajectory is depicted in Fig. 4. The left column shows the reconstruction at the beginning of a trajectory, while the right one the reconstruction 1.5 s later. Since the boxes could be seen in previous measurements, they are properly reconstructed on the right, despite being in the blind spots. It can also be seen that the approach correctly recreates the wall despite only seeing the bottom of it. This is because all the walls have the same height in the training data. This choice was made to reduce the reconstruction to only the relevant features for locomotion. Indeed, the robot cannot overcome larger obstacles, and representing the correct height of such elements does not bring any benefit. Fig. 5 shows the zero-shot estimate of the network on stairs, meaning that the output is computed using only the current measurement. We can conclude that in such scenes, the network can understand the context from spatial data only and generate the correct structure, e.g., the vertical surfaces of the stairs. Of course, adding the temporal information is necessary to reconstruct elements such as those in the left column.

We further analyze the output of the module as a function of the measurement density (Fig. 6). We assess the performance for different amounts of data removed from the measurements for the same validation trajectories. Note that the reported removal rate does not consider the already missing points from the blind spots. The network can cope with up to 50% of data omission, with an F1 score of 88%. The output density then decreases sharply and holes appear, which is reflected by the recall decreasing to a value of 62%. On the other hand, the precision does not decline as much and reaches a value of 82%. When more than 80% of data is omitted, the decoder cannot handle the sparsity and the pruning produces empty tensors. The results show that the network can handle sparse measurements and reconstruct the terrain accurately. It aggregates data on a very local level to produce a bigger picture of the scene. We hypothesize that this is due to the spatial formulation of the problem, where the 3D coordinates of the points in Eq. 1 induce a strong prior about the configuration of the scene. This is in contrast to 2D-based methods on depth images, where the depth information is encoded in the input features. The complete 3D spatial information is not conserved in the sense that a convolution between two neighboring pixels

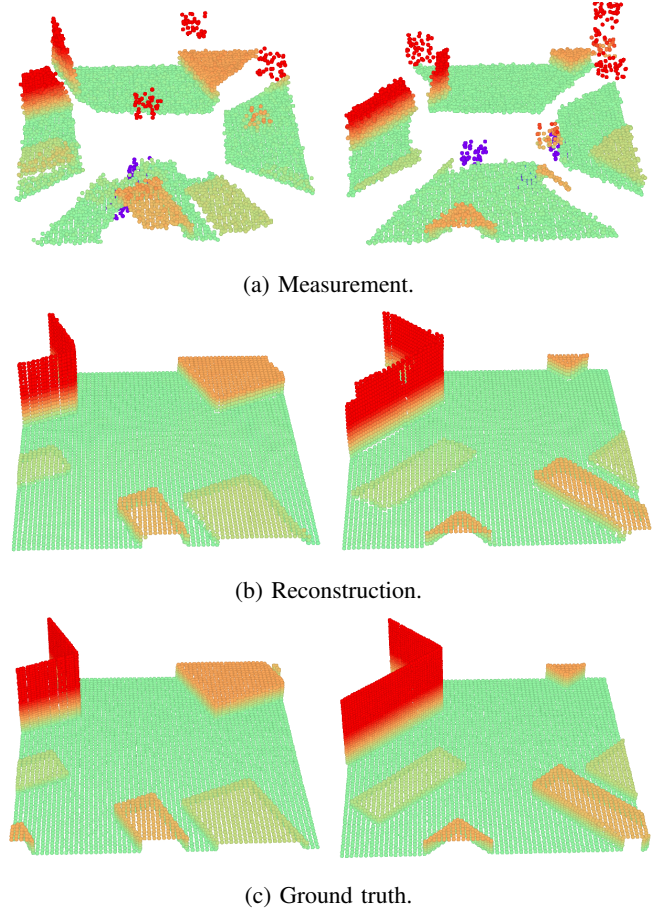


Fig. 4: Reconstruction on a validation trajectory in simulation. The left column shows the estimate at the beginning of the trajectory, and the right the estimate 1.5 s later. While two boxes in the diagonals cannot be seen anymore in the measurements on the right, they are still reconstructed correctly using the evidence from the past.

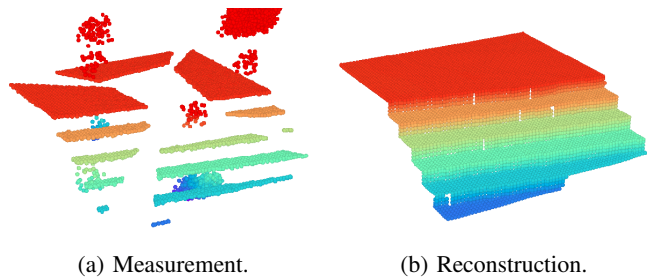


Fig. 5: Zero-shot estimation on stairs, i.e., using only the measurement at the current time-step. The approach can correctly reconstruct the vertical surfaces of the walls using spatial data only.

always occurs, regardless of whether the corresponding points in 3D are very far apart or not.

B. Evaluation on the Robot

The network is deployed on an NVIDIA Jetson Xavier on the robot. A ROS node processes the incoming point cloud

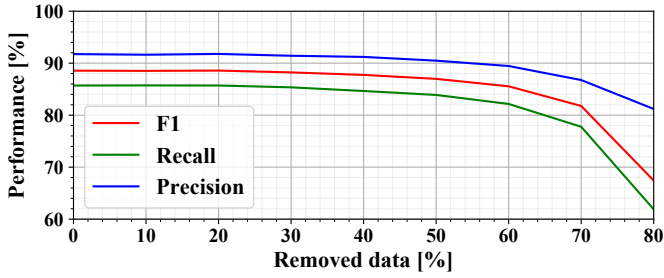


Fig. 6: Mean performance on validation trajectories as a function of the amount of data removed from the measurements. The performance stays constant up to 50% of data removal, after which the recall diminishes rapidly.

TABLE I: Comparison of the different approaches on the box and stairs data sets.

	Method	Pr. [%]	Re. [%]	F1 [%]	MAE [cm]
Stairs	Measurement	87.7	50.7	64.0	0.64
	E.M. [3]	73.2	79.8	76.3	2.1
	Voxblox [7]	76.3	72.3	73.5	1.6
	Ours	86.0	89.9	88.9	0.8
Box	Measurement	80.8	61.6	69.8	1.2
	E.M. [3]	72.2	80.0	75.9	1.7
	Voxblox [7]	74.0	77.9	75.8	1.4
	Ours	84.8	84.9	84.8	1.0

measurements coming from the Intel RealSense cameras at the front, left, right, and back of the robot on a separate thread and maps the data to the GPU for inference. The pose is taken from the state estimator running on the robot. The node then publishes the estimated point cloud, which the locomotion controller uses to query an array of heights around the robot for the policy. Note that during all the experiments, the locomotion controller uses the output of our module for terrain sensing, which shows that it can be deployed in real-world scenarios indoors and outdoors.

Inference on the on-board computer takes on average 70 ms, and the whole node, including the point cloud conversions, inference, and publishing runs at 6 Hz. Due to the computational limits of the hardware, we discard the measurements in between map updates. The reconstructed map around the robot has a dimension of $3.2 \times 3.2 \times 3.2$ m, while the terrain input for the locomotion policy is 1.6×1.0 m. Since the controller runs faster at a rate of 50 Hz, the perceptive inputs for the policy are computed at the pose relative to the latest map. As a result, the robot could walk with a speed of up to 4.8 m/s before reaching the limit of the map. This is more than enough since we run our policy at a maximum speed of 1 m/s.

We evaluate the approach on the real robot in different scenarios and compare the results against the two classical baselines. Both of these methods are computationally efficient and are able to process the data at the same rate as the point cloud measurements are incoming, i.e. 15 Hz. Since these approaches produce meshes, we sample dense point clouds from their terrain estimates and use these for comparison. The ground truth point clouds are generated using the accurate BLK2GO LiDAR scanner. We perform ICP registration between each measurement frame and the BLK2GO map, which accurately estimates the robot's pose relative to the ground truth. In the *Stairs* experiment, the robot walks on

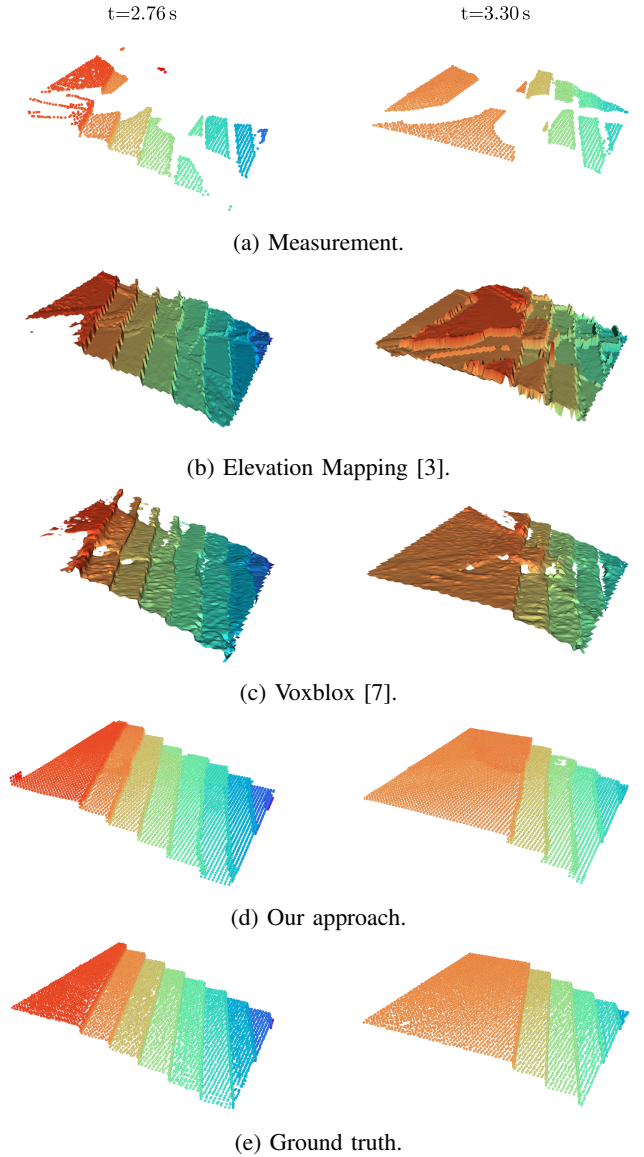


Fig. 7: Comparison of the maps along a trajectory with heavy state estimator drift. During the drift, our approach can detect the mismatch between the previous estimate and the current measurements and immediately aligns the map with the shifted measurements. While Elevation Mapping [3] cannot handle the drift, Voxblox [7] can correct it reasonably well, except immediately below the robot.

stairs of various sizes and textures under different illumination conditions. In the *Box* experiment, the robot walks in a scene with a large box in bright conditions. The wooden surface of the box is challenging for the depth cameras and parts of it are sometimes invisible.

The qualitative results of one of the trajectories in the *Stairs* experiment are shown in Fig. 7. The depth cameras provide good measurements, and the resulting maps produce meaningful results for all three approaches. The reconstructed stairs seem slightly curved on the map borders for all three approaches. This is because of the noise on the edges of the steps, which is stronger on the map's borders, see Fig. 7a. The approaches have difficulties correctly identifying the end

of the step in these regions. Fig. 7d shows that **our approach can contain small holes in the reconstruction. This is due to the pruning formulation and because we do not explicitly train the pipeline to produce a water-tight output.** However, this does not impact the performance of our controller since the holes are very local. As the robot walks up, the state estimator drifts down by 7 cm just after $t = 2.76$ s. This can be seen by comparing the two maps in Fig. 7e. The top surface sinks and changes from red to orange, while it should stay the same color since the maps are expressed in the world frame. **Our approach detects the discrepancy in height between the previous map and the current measurement and immediately shifts down the whole map estimate** (Fig. 7d). **Elevation mapping is not able to cope with the drift and updates the map at the previous height with the new measurements (Fig. 7b), resulting in an uneven map.** On the other hand, **while Voxblox still produces an erroneous reconstruction just below the robot, it is capable of handling the drift and produces a better surface around the robot** (Fig. 7c).

The **quantitative evaluation** of the trajectories is reported in Tab. I. Our method outperforms the other baselines by around 10% in most metrics. The method not only detects more points correctly, but the mean absolute error of the reconstruction is also lower. While the mean absolute error does not seem high for Elevation Mapping, the regions with offsets in the map, such as in Fig. 7b are detrimental for the locomotion policy, as we show in the next subsection. The *Box* dataset is more challenging. The box has a reflective wooden surface and is sometimes invisible to the cameras. Moreover, the jerkier motions of the base represent a challenge for all three approaches. Due to the missing points on the box, it slightly changes dimension over time, see the supplementary video.

Note that our local method compensates for the drift by implicitly aligning the measurements with the map. The map's frame will drift with the robot and not be consistent with the ground truth global frame. However, this does not matter for locomotion, since the map will be correct in the robot's reference frame.

C. Locomotion in Structured Terrains

The previous experiments show that our module can be used by the control policy to overcome urban environments. Compared to Elevation Mapping, the map is cleaner resulting in smoother motions. This can be seen in the video in the supplementary materials, where the robot using the baseline map moves erratically on stairs and the joints shake more. Since the policy is trained with noise in the height measurements, the robot is capable of recovering from the instabilities, but it comes at the cost of worse tracking performance. This makes it challenging for autonomous deployment.

We assess the impact of the map quality on the policy in simulation on a target reaching task for varying amounts of drift. The robot has to cross a challenging terrain within 7 s, and to successfully complete the task, it has to move fast and place the footholds correctly. To model the drift that is varied from 0 cm to 20 cm, bumps of that height are randomly

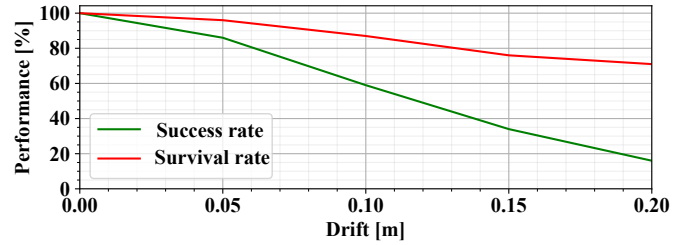


Fig. 8: Performance on a target reaching task in simulation. We compute the average over 4000 trajectories. While the majority of robots do not fall for higher drifts, the tracking performance decreases and the robots fail to reach the target within the allocated time.

placed in the measured terrain. This results in a map that is similar to what we witness on the robot with the other mapping approaches. Fig. 8 shows that while the survival rate only decreases to 70%, the quality of the map highly affects the tracking performance and thus the success rate of the task. For a drift of 20 cm, fewer than 20% of the robots manage to reach the target on time. Additionally, compared to the drift-free case, the robot collides three times more often the knees with the ground with drifts of more than 10 cm.

We deploy our map on a perceptive rough terrain model-based controller [32] to show the generality of our module. This method computes **feasible footholds** on the terrain and tracks them using a model predictive controller. There, having an accurate map is crucial, because an erroneous foothold results in an unexpected trajectory that is hard to recover from, especially on stairs. Drifts in the map often result in failure, even when turning around on the spot in flat terrain, as can be seen in the supplementary video. We show in the video that using our map, the controller is capable of overcoming structured terrains.

V. CONCLUSION

In this work, we presented a terrain representation module that uses noisy and occluded point cloud measurements on a robot to reconstruct the scene faithfully in the robot's vicinity. Our experiments in simulation and on the real robot show that it can successfully accumulate evidence spatially and temporally to build an accurate estimate of the terrain in structured environments. The robot's learning-based and model-based locomotion controllers are able to use our map to walk in an urban setting.

The current approach still has some limitations that have to be tackled. The network can cope with dynamic obstacles reasonably well due to the augmentation with random distractors during training. However, some of the points of the dynamic elements remain in the reconstruction and only vanish slowly. A solution would be to create a data-set containing dynamic obstacles, or use a better noise model.

Also, while the reconstruction is clean most of the time, the depth cameras produce many outliers on the edges of objects. Since that noise is difficult to reproduce in simulation, the data-set could be extended with data from the real robot to make the pipeline ready for full autonomous deployment. Moreover, as

mentioned in the introduction, the unstructured setting is even more challenging. Using data from the robot in such missions is probably key.

In the future, we would like to show the full potential of our method and make the robot walk below tables or climb higher obstacles, which would not be possible with other approaches such as Elevation Mapping. The strong prior induced by the training data could be used to reconstruct the top surface of higher obstacles which might not be visible to the sensor. Also, combining the point cloud data with proprioceptive data such as the position of the feet on contact might be useful when walking on compliant ground such as snow to find out the real ground height. Finally, we would like to explore the direct use of the latent representation of our module by the policy. This could speed up the pipeline by avoiding the decoding stage and give bigger picture of the scene to the policy.

REFERENCES

- [1] D. Belter, P. Łabacki, and P. Skrzypczyński, “Estimating terrain elevation maps from sparse and uncertain multi-sensor data,” in *2012 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2012, pp. 715–722.
- [2] A. Roennau, T. Kerscher, M. Ziegenmeyer, J. Zoellner, and R. Dillmann, “Six-legged walking in rough terrain based on foot point planning,” in *Mobile robotics: solutions and challenges*. World Scientific, 2010, pp. 591–598.
- [3] P. Fankhauser, M. Bloesch, and M. Hutter, “Probabilistic terrain mapping for mobile robots with uncertain localization,” *IEEE Robotics and Automation Letters (RA-L)*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [4] C. Mastalli *et al.*, “Trajectory and foothold optimization using low-dimensional models for rough terrain locomotion,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 1096–1103.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017.
- [6] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.
- [7] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, “Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [8] V. Makoviychuk *et al.*, “Isaac gym: High performance GPU based physics simulation for robot learning,” in *Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [9] D. Kim, J. D. Carlo, B. Katz, G. Bledt, and S. Kim, “Highly dynamic quadrupedal locomotion via whole-body impulse control and model predictive control,” *CoRR*, vol. abs/1909.06586, 2019.
- [10] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, “Sim-to-real: Learning agile locomotion for quadrupedal robots,” in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018.
- [11] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, 2019.
- [12] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [13] J. Siekmann, K. Green, J. Warila, A. Fern, and J. Hurst, “Blind Bipedal Stair Traversal via Sim-to-Real Reinforcement Learning,” in *Proceedings of Robotics: Science and Systems*, July 2021.
- [14] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *5th Annual Conference on Robot Learning*, 2021.
- [15] S. Gangapurwala, M. Geisert, R. Orsolino, M. F. Fallon, and I. Havoutis, “Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control,” *ArXiv*, vol. abs/2012.03094, 2020.
- [16] R. Yang, M. Zhang, N. Hansen, H. Xu, and X. Wang, “Learning vision-guided quadrupedal locomotion end-to-end with cross-modal transformers,” in *International Conference on Learning Representations*, 2022.
- [17] G. Klein and D. Murray, “Parallel tracking and mapping for small ar workspaces,” in *2007 6th IEEE and ACM international symposium on mixed and augmented reality*. IEEE, 2007, pp. 225–234.
- [18] C. Forster, M. Pizzoli, and D. Scaramuzza, “Svo: Fast semi-direct monocular visual odometry,” in *2014 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2014, pp. 15–22.
- [19] J. Engel, T. Schöps, and D. Cremers, “Lsd-slam: Large-scale direct monocular slam,” in *European conference on computer vision*. Springer, 2014, pp. 834–849.
- [20] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, 2013.
- [21] D. Kim, D. Carballo, J. Di Carlo, B. Katz, G. Bledt, B. Lim, and S. Kim, “Vision aided dynamic exploration of unstructured terrain with a small-scale quadruped robot,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 2464–2470.
- [22] R. O. Chavez-Garcia, J. Guzzi, L. M. Gambardella, and A. Giusti, “Learning ground traversability from simulations,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1695–1702, 2018.
- [23] M. Stölzle, T. Miki, L. Gerdes, M. Azkarate, and M. Hutter, “Reconstructing occluded elevation information in terrain maps with self-supervised learning,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1697–1704, 2022.
- [24] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger, “Real-time 3d reconstruction at scale using voxel hashing,” *ACM Transactions on Graphics (ToG)*, vol. 32, no. 6, pp. 1–11, 2013.
- [25] S. Song, F. Yu, A. Zeng, A. X. Chang, M. Savva, and T. Funkhouser, “Semantic scene completion from a single depth image,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1746–1754.
- [26] A. Dai, D. Ritchie, M. Bokeloh, S. Reed, J. Sturm, and M. Nießner, “Scancomplete: Large-scale scene completion and semantic segmentation for 3d scans,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4578–4587.
- [27] J. Choe, S. Im, F. Rameau, M. Kang, and I. S. Kweon, “Volumefusion: Deep depth fusion for 3d scene reconstruction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 16 086–16 095.
- [28] J. Sun, Y. Xie, L. Chen, X. Zhou, and H. Bao, “Neuralrecon: Real-time coherent 3d reconstruction from monocular video,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 15 598–15 607.
- [29] J. Gwak, C. B. Choy, and S. Savarese, “Generative sparse detection networks for 3d single-shot object detection,” in *European conference on computer vision*, 2020.
- [30] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, 2015, pp. 234–241.
- [31] C. Choy, J. Gwak, and S. Savarese, “4d spatio-temporal convnets: Minkowski convolutional neural networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3075–3084.
- [32] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, “Perceptive locomotion through nonlinear model predictive control,” (submitted to) *IEEE Transactions on Robotics*, 2022.